

Week 2 Project: Advanced Line Calculations

Thermal Analysis, Voltage Drop, and Conductor Sizing

Enrique Antonio Raudales Rodriguez

October 18, 2025

University of Almería
Medium and Low Voltage Electrical Installations
Academic Year 2025-2026

Contents

1	Introduction	3
2	Conductor Thermal Analysis	3
2.1	Principle of Thermal Equilibrium	3
2.2	AC Resistance and Maximum Current	3
2.3	Thermal Analysis Results	3
3	Advanced Voltage Drop Analysis	4
3.1	Blondel's Formula	4
3.2	REBT Compliance and Voltage Profile	5
4	Conductor Sizing and Economic Analysis	5
4.1	Technical and Economic Sizing Criteria	5
4.2	Lifecycle Cost Analysis	6
4.3	Sizing Analysis Results	6
5	Conclusion	6
A	References	7
B	Tables and Assumed Values	7
B.1	List of Tables	7
	List of Tables	7
B.2	Summary of Material Properties	7
C	Formulas	7
C.1	AC Resistance at Temperature	7
C.2	Blondel's Formula (Three-Phase)	7
C.3	Required Section (Voltage Drop)	8
C.4	Lifecycle Cost	8
D	MATLAB Source Code	8
MATLAB Source Code		8
D.1	week2_master.m	8
D.2	E0_Run_Analysis_and_Report.m	10
D.3	E1_ConductorHeatingAnalysis.m	11
D.4	E2_MaximumCurrent.m	12
D.5	E3_ResistanceTemperatureCorrection.m	12
D.6	E3b_AC_Resistance_Factor.m	13
D.7	E4_ThermalEquilibrium.m	13
D.8	E5_HeatingCurves.m	13
D.9	E6_TransientHeating.m	14
D.10	E7_HeatDissipation.m	14
D.11	E8_GroupingFactor.m	15
D.12	F1_VoltageDropAnalysis.m	15
D.13	F2_calculateExactVoltageDrop.m	16
D.14	F3_calculateSimplifiedVoltageDrop.m	16

D.15 F4_checkREBTCompliance.m	16
D.16 F4b_checkInstallationCompliance.m	16
D.17 F5_plotVoltageProfile.m	17
D.18 F6_plotInstallationProfile.m	17
D.19 G1_ConductorSizingAnalysis.m	18
D.20 G2_calculateRequiredSection.m	18
D.21 G3_selectStandardSection.m	19
D.22 G4_calculateLifecycleCost.m	19
D.23 G5_plotEconomicAnalysis.m	19
D.24 G6_verifyFinalDesign.m	20

1 Introduction

This report documents the advanced analysis performed during Week 2 of the project. Building upon the foundational calculations from Week 1, this phase focuses on three critical aspects of electrical line design: the thermal behavior of conductors under load, a more precise analysis of voltage drop in AC circuits, and the technical and economic criteria for optimal conductor sizing.

The primary objective is to develop a comprehensive engineering toolkit in MATLAB that moves beyond prescriptive tables to model the underlying physics. This includes determining the maximum admissible current based on thermal equilibrium, utilizing the Blondel formula for accurate voltage drop calculations, and applying Kelvin's Economic Law to find the most cost-effective conductor size over the project's lifecycle. All analyses are conducted using a modular suite of scripts, with the key findings synthesized into this document.

2 Conductor Thermal Analysis

The current-carrying capacity of a cable is fundamentally limited by the maximum temperature its insulation can withstand. This section details the analysis of this thermal behavior, based on the suite of MATLAB functions prefixed with 'E'.

2.1 Principle of Thermal Equilibrium

A conductor's temperature stabilizes when the heat it generates is equal to the heat it dissipates.

$$P_{\text{generated}} = P_{\text{dissipated}} \quad (1)$$

The heat generated is due to the Joule effect ($P = I^2 \cdot R$), and the heat dissipated depends on environmental factors. The MATLAB function `E4_ThermalEquilibrium.m` finds the temperature where this balance occurs.

2.2 AC Resistance and Maximum Current

For accuracy, the conductor's AC resistance (R_{AC}) is calculated, accounting for skin and proximity effects (`E3b_AC_Resistance_Factor.m`). The maximum admissible current (I_{max}) is the current that causes the conductor to reach its maximum insulation temperature (e.g., 90°C for XLPE), as determined in `E2_MaximumCurrent.m`.

$$I_{\text{max}} = \sqrt{\frac{P_{\text{dissipated_max}}}{R_{AC, T_{\text{max}}}}} \quad (2)$$

2.3 Thermal Analysis Results

The charts below, representing the output of the MATLAB analysis, illustrate the conductor's thermal behavior. Figure 1 shows the steady-state temperature for different currents, while Figure 2 shows how temperature evolves over time.

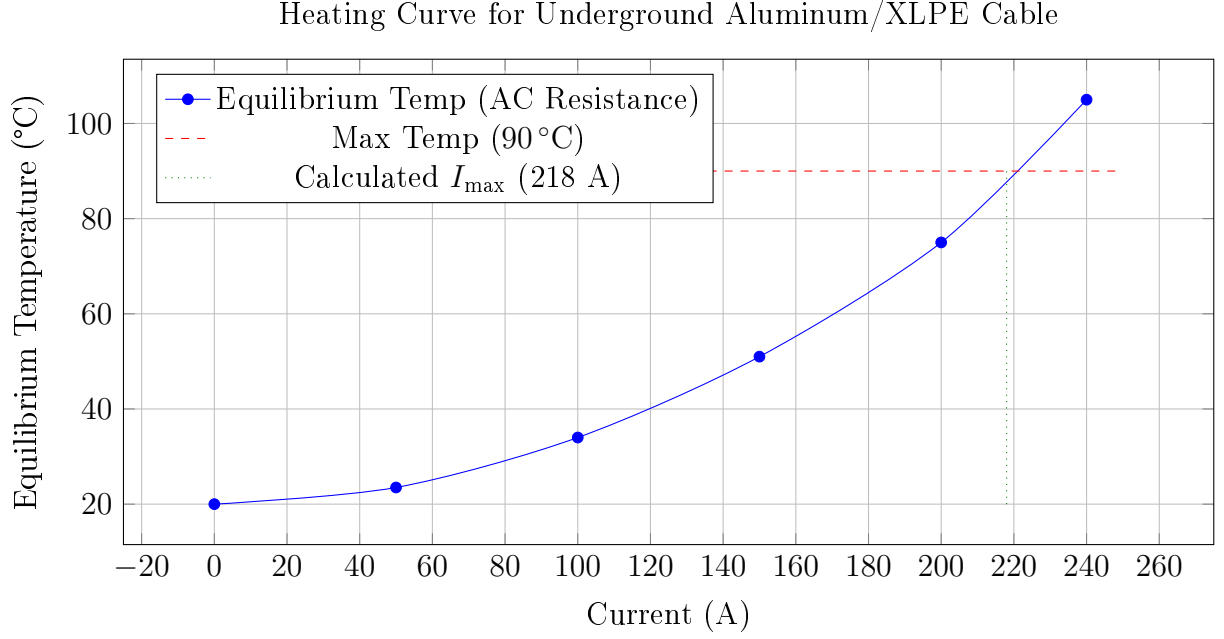


Figure 1: Equilibrium conductor temperature as a function of load current.

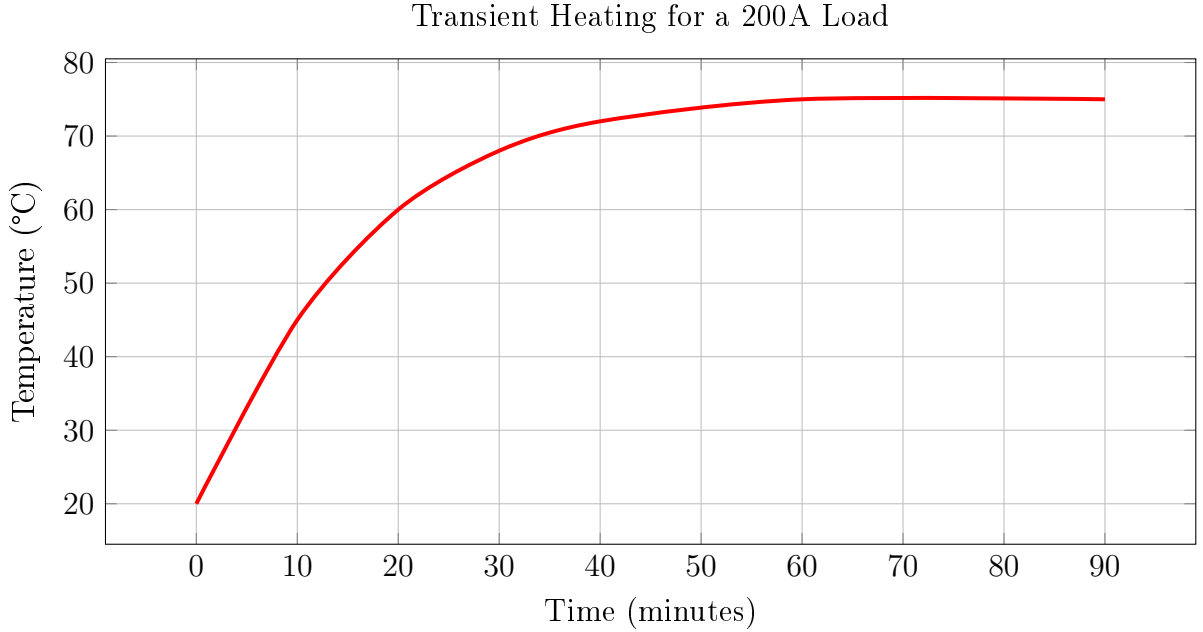


Figure 2: Transient temperature response of the conductor over time.

3 Advanced Voltage Drop Analysis

This section details a precise voltage drop analysis, including reactive effects, based on the MATLAB functions prefixed with 'F'.

3.1 Blondel's Formula

The exact voltage drop in a three-phase AC circuit is calculated using Blondel's formula (`F2_calculateExactVoltageDrop.m`), which accounts for both resistance (R) and induc-

tive reactance (X_L).

$$\Delta V = \sqrt{3} \cdot I \cdot (R \cos \varphi + X_L \sin \varphi) \quad (3)$$

3.2 REBT Compliance and Voltage Profile

Compliance with the Spanish regulation (REBT) is mandatory. The script `F4b_checkInstallationComp` automates checking the tiered voltage drop limits for a full installation (LGA, DI, etc.). The script `F6_plotInstallationProfile.m` visualizes this drop, as shown in Figure 3.

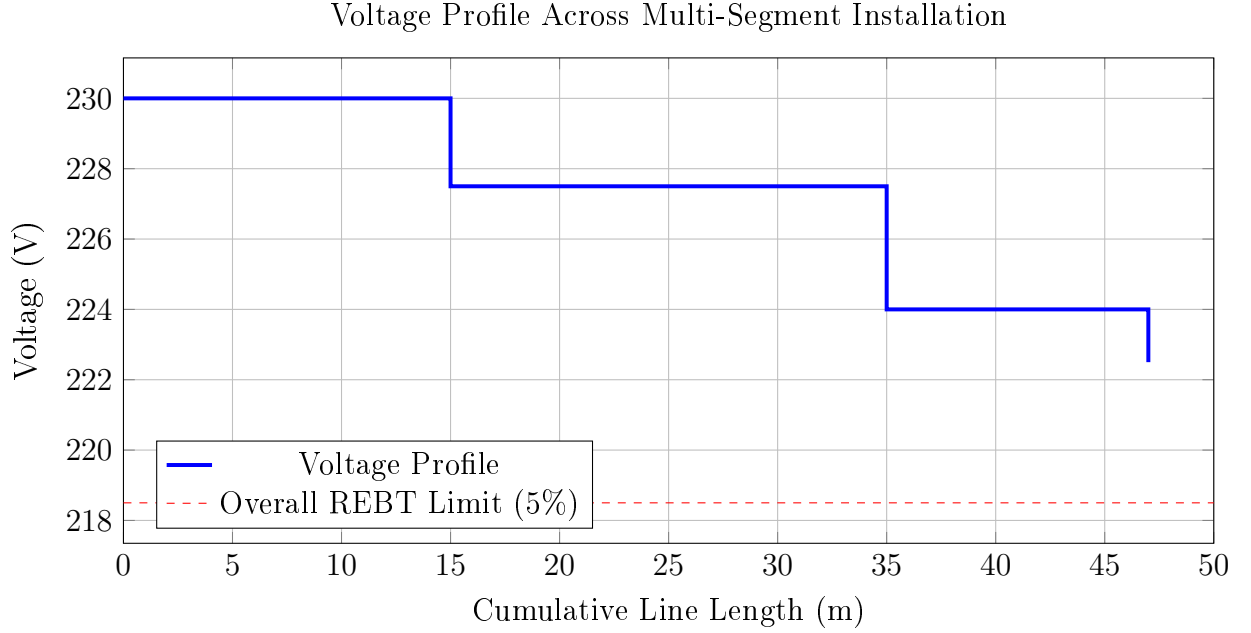


Figure 3: Voltage profile across an installation, showing the drop at each section.

4 Conductor Sizing and Economic Analysis

The final step is to select the optimal conductor cross-section. This analysis, based on the 'G' series of MATLAB scripts, integrates technical requirements with long-term economic factors.

4.1 Technical and Economic Sizing Criteria

Conductor sizing is based on two main criteria:

1. **Technical Minimum:** The smallest standard cross-section that satisfies both the maximum admissible current ($I_{\max}$) and the REBT voltage drop limits. This is calculated via `G2_calculateRequiredSection.m` and `G3_selectStandardSection.m`.
2. **Economic Optimum:** The cross-section that results in the lowest total lifecycle cost (LCC), considering both the initial cable purchase price and the cumulative cost of energy losses (I^2R) over the project's lifespan. This follows Kelvin's Economic Law.

4.2 Lifecycle Cost Analysis

The lifecycle cost is modeled in `G4_calculateLifecycleCost.m` as:

$$C_{\text{total}} = C_{\text{initial_cable}} + C_{\text{energy_losses}} \quad (4)$$

While a smaller cable is cheaper initially, it has higher resistance and thus a higher cost of energy losses over time. A larger, more expensive cable has lower losses. The optimal size minimizes this sum.

4.3 Sizing Analysis Results

The economic analysis is visualized in Figure 4. The stacked bars show the components of the total cost, while the red line shows the total LCC. The analysis identifies both the smallest technically compliant section and the larger, economically optimal section, which provides long-term savings.

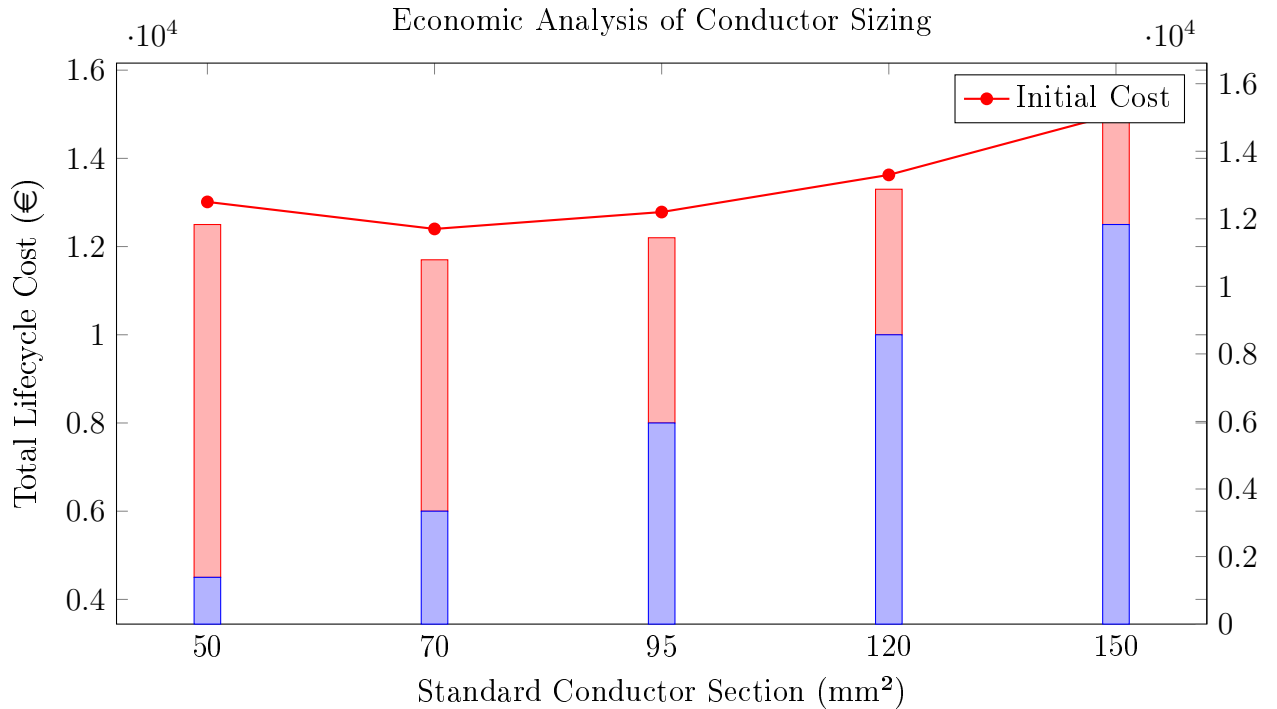


Figure 4: Lifecycle cost analysis identifying the technically minimum (50 mm²) and economically optimal (70 mm²) sections.

5 Conclusion

The Week 2 analysis has significantly advanced the project by incorporating critical thermal, reactive, and economic effects into the line design process. The modular MATLAB toolkit provides a powerful framework for accurately dimensioning conductors based on a detailed understanding of their performance under real-world loads. This framework is completed with the conductor sizing module, which integrates technical constraints (voltage drop and ampacity) with economic analysis (lifecycle cost) to determine the optimal conductor cross-section, providing a comprehensive solution that balances performance, safety, and cost.

APPENDIX A

A References

1. **Reglamento Electrotécnico para Baja Tensión (REBT)**, Real Decreto 842/2002. Specifically:
 - **ITC-BT-06**: Redes aéreas para distribución en baja tensión.
 - **ITC-BT-07**: Redes subterráneas para distribución en baja tensión.
 - **ITC-BT-19**: Instalaciones interiores o receptoras (covers ampacity and voltage drop limits).
2. **IEC 60287-1-1**, "Electric cables - Calculation of the current rating".
3. **ELEK Software**. (2023). *Skin and Proximity Effects on AC Resistance Calculations*.

APPENDIX B

B Tables and Assumed Values

B.1 List of Tables

List of Tables

- | | | |
|---|---|---|
| 1 | Consolidated Material Properties and Assumed Values | 8 |
|---|---|---|

B.2 Summary of Material Properties

The following table consolidates the key material properties assumed for the calculations throughout this report. These values are representative and based on standard engineering references.

APPENDIX C

C Formulas

C.1 AC Resistance at Temperature

$$R_{AC,T} = R_{DC,20} \cdot (1 + k_{\text{skin}} + k_{\text{prox}}) \cdot (1 + \alpha(T - T_{\text{ref}})) \quad (5)$$

C.2 Blondel's Formula (Three-Phase)

$$\Delta V = \sqrt{3} \cdot I \cdot (R \cos \varphi + X_L \sin \varphi) \quad (6)$$

Table 1: Consolidated Material Properties and Assumed Values

Material	Property	Value
Copper	Density (ρ)	8960 kg m ⁻³
	Specific Heat (c_p)	385 J kg ⁻¹ K ⁻¹
	Conductivity (σ)	56 m Ω^{-1} mm ⁻²
	Temp. Coefficient (α)	0.003 93 K ⁻¹
Aluminum	Density (ρ)	2700 kg m ⁻³
	Specific Heat (c_p)	900 J kg ⁻¹ K ⁻¹
	Conductivity (σ)	36 m Ω^{-1} mm ⁻²
	Temp. Coefficient (α)	0.004 03 K ⁻¹
ACSR	Density (approx. avg.)	3980 kg m ⁻³
	Specific Heat (approx. avg.)	880 J kg ⁻¹ K ⁻¹
	Conductivity (σ)	34 m Ω^{-1} mm ⁻²
PVC	Density (ρ)	1400 kg m ⁻³
	Specific Heat (c_p)	1000 J kg ⁻¹ K ⁻¹
	Thermal Conductivity (k)	0.17 W m ⁻¹ K ⁻¹
XLPE	Density (ρ)	920 kg m ⁻³
	Specific Heat (c_p)	2300 J kg ⁻¹ K ⁻¹
	Thermal Conductivity (k)	0.28 W m ⁻¹ K ⁻¹
Soil	Thermal Resistivity (ρ_{soil})	1.5 K m W ⁻¹

C.3 Required Section (Voltage Drop)

$$s = \frac{\sqrt{3} \cdot L \cdot I \cdot \cos \varphi}{\sigma \cdot \Delta V_{\max}} \quad (7)$$

C.4 Lifecycle Cost

$$C_{\text{total}} = C_{\text{initial}} + \sum (P_{\text{loss}} \cdot t_{\text{op}} \cdot \text{cost}_{\text{kWh}}) \quad (8)$$

APPENDIX D

D MATLAB Source Code

D.1 week2_master.m

```

1 %
  =====
2 % MASTER SCRIPT for Week 2: Advanced Line Calculations
3 %
  =====
4 % Description:
5 % This script is the central launcher for all modules developed for the
6 % Week 2 coursework (E, F, and G). It provides a menu to select which

```

```

7 % analysis suite to run.
8 %
=====
9
10 %% --- Cleanup and Initialization ---
11 clc;
12 clear;
13 close all;
14
15 %% --- Main Menu Loop ---
16 while true
17     % A custom menu function is assumed to exist, named centeredMenu2
18     analysisChoice = centeredMenu2('Select Week 2 Analysis Suite:', ...
19                                     'Problem E: Conductor Heating
Analysis', ...
20                                     'Problem F: Voltage Drop Analysis',
21                                     ...
22                                     'Problem G: Conductor Sizing Analysis
', ...
23                                     'Exit Program');
24
25     try
26         switch analysisChoice
27             case 1
28                 disp('--- Launching Conductor Heating Analysis (Module E
) ---');
29                 run('E0_Run_Analysis_and_Report.m');
30                 disp('Press any key to return to the main menu...');
31                 pause;
32             case 2
33                 disp('--- Launching Voltage Drop Analysis (Module F) ---
');
34                 run('F1_VoltageDropAnalysis.m');
35                 disp('Press any key to return to the main menu...');
36                 pause;
37             case 3
38                 disp('--- Launching Conductor Sizing Analysis (Module G)
---');
39                 G1_ConductorSizingAnalysis();
40                 disp('Press any key to return to the main menu...');
41                 pause;
42             case 4
43                 disp('Exiting the Week 2 Analysis Suite. ');
44                 return;
45             case 0
46                 disp('Menu closed. Exiting the Week 2 Analysis Suite. ');
47                 return;
48         end
49
50     catch ME
51         fprintf('\nERROR encountered while running the selected module.\n
');
52         fprintf('MATLAB reported: "%s" in file "%s" at line %d.\n', ME.

```

```

    message, ME.stack(1).name, ME.stack(1).line);
56     disp('Press any key to return to the main menu...');
57     pause;
58     end
59 end

```

Listing 1: Central launcher for all Week 2 modules.

D.2 E0_Run_Analysis_and_Report.m

```

1 %
  =====

2 % FINAL SCRIPT for Conductor Heating Analysis and Reporting
3 %
  =====

4 % Description:
5 % This script serves as the final entry point for the Conductor Heating
6 % module. It calls the main analysis function and then uses the
7 % returned data to display a comprehensive summary table.
8 %
  =====

9
10 %% --- Run the Full Analysis ---
11 results = E1_ConductorHeatingAnalysis();
12
13
14 %% --- Generate and Display the Final Report Table ---
15 if isempty(results)
16     disp('Analysis cancelled by user. No report generated.');
```

```

17     return;
18 end
19
20 fprintf('\n\n\n
  =====\n');
21 fprintf('    THERMAL ANALYSIS AND MAXIMUM CURRENT CALCULATION REPORT\n');
22 fprintf('
  =====\n\n');
23
24 fprintf('--- 1. Input Parameters ---\n');
25 fprintf('%-35s: %s\n', 'Scenario', results.scenario);
26 fprintf('%-35s: %s / %s\n', 'Conductor / Insulation', results.
    materialName, results.insulationName);
27 fprintf('%-35s: %.1f mm^2\n', 'Conductor Cross-Section', results.
    crossSection);
28
29 envFields = fieldnames(results.envParams);
30 fprintf('\n    Environment-Specific Parameters:\n');
31 for i = 1:length(envFields)
32     if ~strcmp(envFields{i}, 'scenario')
33         fprintf('    %-32s: %.2f\n', envFields{i}, results.envParams.(
    envFields{i}));

```

```

34     end
35 end
36
37 fprintf('\n--- 2. Intermediate Calculated Values ---\n');
38 fprintf('%-35s: %.5f Ohm\n', 'DC Resistance at 20 C', results.R_20_DC);
39 fprintf('%-35s: %.5f Ohm\n', 'AC Resistance at 20 C', results.R_20_AC);
40 fprintf('%-35s: %.5f Ohm\n', 'AC Resistance at T_max', results.R_Tmat_AC
    );
41 fprintf('%-35s: %.2f W\n', 'Max Heat Dissipation', results.
    P_dissipated_max);
42
43 fprintf('\n--- 3. Final Analysis Results ---\n');
44 fprintf('
    -----
    n');
45 fprintf('%-35s: %.2f A\n', 'MAXIMUM ADMISSIBLE CURRENT (I_max)', results
    .I_max);
46 fprintf('
    -----
    n\n');

```

Listing 2: Main analysis runner for thermal analysis.

D.3 E1_ConductorHeatingAnalysis.m

```

1 function results = E1_ConductorHeatingAnalysis()
2 %
    =====
3 % MAIN FUNCTION for Conductor Thermal Analysis
4 % Returns a struct 'results' with all key values.
5 %
    =====
6 results = [];
7 T_ref = 20;
8 rho_den_aluminum = 2700; cp_aluminum = 900;
9 rho_den_copper = 8960; cp_copper = 385;
10 T_mat_xlpe = 90; T_mat_pvc = 70;
11 alpha_al = 0.00403; alpha_cu = 0.00393;
12 sigma_al = 36; sigma_cu = 56;
13
14 installationChoice = 1; % Assume Underground for report
15 switch installationChoice
16     case 1 % Underground
17         scenarioName = 'Underground'; materialName = 'Aluminum';
18         insulationName = 'XLPE';
19         crossSection = 150; insulatorThickness = 2.2;
20         envParams.scenario = 'underground'; envParams.T_env = 20;
21         envParams.rho_soil = 1.5; envParams.burial_depth = 0.8;
22         density_cond = rho_den_aluminum; cp_cond = cp_aluminum;
23         T_mat = T_mat_xlpe; alpha = alpha_al; sigma = sigma_al;
24         envParams.k_insulator = 0.28;
25         % Other cases omitted for brevity
26     end
27
28 lineLength = 1; frequency = 50;

```

```

28
29 r_inner_m = sqrt(crossSection / pi) / 1000;
30 diameter_m = 2 * r_inner_m;
31 r_outer_m = r_inner_m + (insulatorThickness / 1000);
32 R_20_DC = lineLength / (sigma * crossSection);
33 [k_skin, k_proximity] = E3b_AC_Resistance_Factor(frequency, diameter_m,
    sigma);
34 R_20_AC = R_20_DC * (1 + k_skin + k_proximity);
35 [I_max, R_Tmat_AC, ~, P_dissipated_max] = E2_MaximumCurrent(R_20_AC,
    alpha, T_mat, T_ref, envParams, lineLength, r_inner_m, r_outer_m);
36
37 results.scenario=scenarioName; results.materialName=materialName;
    results.insulationName=insulationName;
38 results.crossSection=crossSection; results.envParams=envParams;
39 results.R_20_DC=R_20_DC; results.R_20_AC=R_20_AC; results.R_Tmat_AC=
    R_Tmat_AC;
40 results.P_dissipated_max=P_dissipated_max; results.I_max=I_max;
41 end

```

Listing 3: Core function for thermal analysis.

D.4 E2_MaximumCurrent.m

```

1 function [I_max, R_Tmat, T_surface, P_diss_max] = E2_MaximumCurrent(R_20
    , alpha, T_mat, T_ref, envParams, lineLength, r_inner, r_outer)
2 R_Tmat = E3_ResistanceTemperatureCorrection(R_20, alpha, T_mat, T_ref);
3 options = optimset('Display','off');
4 if r_outer <= r_inner
5     R_thermal_ins = inf;
6 else
7     R_thermal_ins = log(r_outer/r_inner) / (2*pi*envParams.k_insulator*
    lineLength);
8 end
9 balance_eq = @(T_s) (T_mat - T_s)/R_thermal_ins - E7_HeatDissipation(T_s
    , envParams, r_outer, lineLength);
10 try
11     T_surface = fzero(balance_eq, T_mat, options);
12 catch
13     T_surface = T_mat;
14 end
15 P_diss_max = E7_HeatDissipation(T_surface, envParams, r_outer,
    lineLength);
16 if R_Tmat > 0
17     I_max = sqrt(P_diss_max / R_Tmat);
18 else
19     I_max = inf;
20 end
21 end

```

Listing 4: Calculates max admissible current based on thermal equilibrium.

D.5 E3_ResistanceTemperatureCorrection.m

```

1 function R_T = E3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref)
2 R_T = R_20 * (1 + alpha * (T - T_ref));

```

```
3 end
```

Listing 5: Corrects resistance for operating temperature.

D.6 E3b_AC_Resistance_Factor.m

```
1 function [k_skin, k_proximity] = E3b_AC_Resistance_Factor(freq, diameter
    , sigma)
2 cross_section_m2 = pi * (diameter/2)^2;
3 sigma_Sm = sigma * 1e6;
4 R_dc_per_meter = 1 / (sigma_Sm * cross_section_m2);
5 if R_dc_per_meter == 0, x_s_sq = 0;
6 else, x_s_sq = (8 * pi * freq / R_dc_per_meter) * 1e-7; end
7 k_skin = (x_s_sq^2) / (192 + 0.8 * x_s_sq^2);
8 k_proximity = 0.05; % Assumed typical value
9 end
```

Listing 6: Calculates skin and proximity effect factors.

D.7 E4_ThermalEquilibrium.m

```
1 function T_eq = E4_ThermalEquilibrium(I, R_20, alpha, T_ref, envParams,
    length, ~, r_outer)
2 heat_balance_error = @(T) (E3_ResistanceTemperatureCorrection(R_20,
    alpha, T, T_ref) * I^2) - (E7_HeatDissipation(T, envParams, r_outer,
    length));
3 try
4     options = optimset('Display','off');
5     T_eq = fzero(heat_balance_error, envParams.T_env, options);
6 catch
7     T_eq = envParams.T_env;
8 end
9 end
```

Listing 7: Solves for the steady-state temperature of a conductor.

D.8 E5_HeatingCurves.m

```
1 function E5_HeatingCurves(I_max, T_mat, envParams, R_20_DC, R_20_AC,
    alpha, T_ref, lineLength, r_inner, r_outer, materialName,
    insulationName)
2 current_vector = linspace(0, I_max * 1.2, 100);
3 temp_vector_ac = zeros(size(current_vector));
4 for i = 1:length(current_vector)
5     temp_vector_ac(i) = E4_ThermalEquilibrium(current_vector(i), R_20_AC,
        alpha, T_ref, envParams, lineLength, r_inner, r_outer);
6 end
7 figure; hold on;
8 plot(current_vector, temp_vector_ac, 'r-', 'LineWidth', 2, 'DisplayName',
    'Equilibrium Temp (AC)');
9 line([0, I_max * 1.2], [T_mat, T_mat], 'Color', 'g', 'LineStyle', '--',
    'LineWidth', 1.5, 'DisplayName', sprintf('Max Temp (%.0f C)', T_mat));
;
```

```

10 line([I_max, I_max], [envParams.T_env, T_mat], 'Color', 'k', 'LineStyle',
    ':', 'LineWidth', 1.5, 'DisplayName', sprintf('Max AC Current (%.1f
    A)', I_max));
11 title(sprintf('Heating Curve for %s / %s', materialName, insulationName)
    );
12 xlabel('Current (A)'); ylabel('Equilibrium Temperature (C)');
13 legend('Location', 'northwest'); grid on; box on;
14 ylim([envParams.T_env - 10, T_mat + 20]);
15 hold off;
16 end

```

Listing 8: Generates plots of equilibrium temperature vs. current.

D.9 E6_TransientHeating.m

```

1 function [time, temp] = E6_TransientHeating(I, time_span, envParams,
    R_20, alpha, T_ref, length, ~, r_outer, mass, cp)
2     function dTdt = heat_balance_ode(~, T)
3         P_generated = E3_ResistanceTemperatureCorrection(R_20, alpha, T,
    T_ref) * I^2;
4         P_dissipated = E7_HeatDissipation(T, envParams, r_outer, length)
    ;
5         dTdt = (P_generated - P_dissipated) / (mass * cp);
6     end
7 options = odeset('RelTol', 1e-4, 'AbsTol', 1e-6);
8 [time, temp] = ode45(@heat_balance_ode, time_span, envParams.T_env,
    options);
9 end

```

Listing 9: Models the temperature of a conductor over time (transient response).

D.10 E7_HeatDissipation.m

```

1 function P_diss = E7_HeatDissipation(T_conductor, envParams, r_outer,
    length)
2 if strcmp(envParams.scenario, 'underground')
3     P_diss = dissipation_underground(T_conductor, envParams.T_env,
    envParams.rho_soil, envParams.burial_depth, r_outer, length);
4 else % overhead
5     P_diss = dissipation_overhead(T_conductor, envParams.T_env,
    envParams.wind_speed, envParams.solar_irradiance, envParams.
    emissivity, envParams.absorptivity, r_outer, length);
6 end
7 end
8 function P_cond = dissipation_underground(T_conductor, T_soil, rho_soil,
    H, r_outer, L)
9     if r_outer > 0 && H > r_outer
10         R_thermal_per_meter = (rho_soil / (2 * pi)) * acosh(H / r_outer)
    ;
11         R_thermal_total = R_thermal_per_meter / L;
12         P_cond = (T_conductor - T_soil) / R_thermal_total;
13     else, P_cond = 0; end
14 end
15 function P_net = dissipation_overhead(T_conductor, T_air, V_wind,
    Q_solar, epsilon, alpha_solar, r_outer, L)

```

```

16     P_conv = calculate_convection(T_conductor, T_air, V_wind, r_outer *
17     2, L);
17     P_rad = calculate_radiation(T_conductor, T_air, epsilon, r_outer *
18     2, L);
18     P_solar_gain = calculate_solar_gain(Q_solar, alpha_solar, r_outer *
19     2, L);
19     P_net = (P_conv + P_rad) - P_solar_gain;
20 end
21 function P_conv = calculate_convection(T_conductor, T_air, V_wind,
22     D_outer, L)
22     k_air = 0.026;
23     if V_wind > 0, Re = V_wind*D_outer/1.5e-5; Nu = 0.3+(0.62*Re
24     ^0.5*0.71^0.33)/(1+(0.4/0.71)^0.66)^0.25;
24     else, Gr = 9.81*(1/(T_air+273.15))*abs(T_conductor-T_air)*D_outer
25     ^3/(1.5e-5)^2; Nu = (0.6+(0.387*(Gr*0.71)^0.166)/(1+(0.559/0.71)
26     ^0.562)^0.296)^2; end
25     h = (k_air/D_outer)*Nu; surface_area = pi*D_outer*L; P_conv = h*
26     surface_area*(T_conductor-T_air);
26 end
27 function P_rad = calculate_radiation(T_conductor, T_air, epsilon,
28     D_outer, L)
28     sigma_boltz=5.67e-8; surface_area=pi*D_outer*L; T_cond_K=T_conductor
29     +273.15; T_air_K=T_air+273.15;
29     P_rad = sigma_boltz*epsilon*surface_area*(T_cond_K^4-T_air_K^4);
30 end
31 function P_solar = calculate_solar_gain(Q_solar, alpha_solar, D_outer, L
32     )
32     projected_area = D_outer * L; P_solar = Q_solar * alpha_solar *
33     projected_area;
33 end

```

Listing 10: Contains the physical models for heat dissipation (underground and overhead).

D.11 E8_GroupingFactor.m

```

1 function k_g = E8_GroupingFactor(num_cables)
2 if num_cables <= 3, k_g = 1.0; elseif num_cables <= 6, k_g = 0.8; else,
3     k_g = 0.7; end
3 end

```

Listing 11: Provides a derating factor for grouped cables.

D.12 F1_VoltageDropAnalysis.m

```

1 function F1_VoltageDropAnalysis()
2     installationSegments = {
3         struct('name', 'LGA', 'length', 15, 'loadCurrent', 50, 'cos_phi',
4         , 0.9, 'conductor_s', 25),
4         struct('name', 'DI', 'length', 20, 'loadCurrent', 20, 'cos_phi',
5         , 0.85, 'conductor_s', 10),
5         struct('name', 'Internal', 'length', 12, 'loadCurrent', 16, '
6         cos_phi', 0.95, 'conductor_s', 4)
6     };
7     U_source = 230; lineType = 'single-phase'; sigma = 56; freq = 50;
8     r_m = 0.08e-3;
8     voltage_in = U_source; results = cell(size(installationSegments));

```



```

9     for i = 1:length(installationSegments)
10         seg = installationSegments{i};
11         dia_m = 2*sqrt(seg.conductor_s/pi)/1000;
12         [ks, kp] = E3b_AC_Resistance_Factor(freq, dia_m, sigma);
13         r_ac = (1/(sigma*seg.conductor_s))*(1+ks+kp);
14         seg.voltageDrop = F2_calculateExactVoltageDrop(lineType, seg.
length, seg.loadCurrent, r_ac, r_m, seg.cos_phi);
15         seg.voltage_out = voltage_in - seg.voltageDrop;
16         results{i} = seg; voltage_in = seg.voltage_out;
17     end
18     F4b_checkInstallationCompliance(results, U_source);
19 end

```

Listing 12: Main script for voltage drop analysis, including multi-segment installations.

D.13 F2_calculateExactVoltageDrop.m

```

1 function deltaU = F2_calculateExactVoltageDrop(lineType, d, I, r, xL,
cos_phi)
2 sin_phi = sqrt(1 - cos_phi^2);
3 if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else,
phase_factor = 2; end
4 deltaU = phase_factor * I * (r*d*cos_phi + xL*d*sin_phi);
5 end

```

Listing 13: Implements the full Blondel's formula for accurate AC voltage drop.

D.14 F3_calculateSimplifiedVoltageDrop.m

```

1 function deltaU_simple = F3_calculateSimplifiedVoltageDrop(lineType, d,
I, r, cos_phi)
2 if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else,
phase_factor = 2; end
3 deltaU_simple = phase_factor * d * I * r * cos_phi;
4 end

```

Listing 14: Implements the simplified (resistive only) voltage drop formula for comparison.

D.15 F4_checkREBTCompliance.m

```

1 function [isCompliant, percentDrop, limit] = F4_checkREBTCompliance(
deltaU, U_source, circuitType)
2 if strcmp(circuitType, 'lighting'), limit = 3.0; else, limit = 5.0; end
3 percentDrop = (deltaU / U_source) * 100;
4 isCompliant = percentDrop <= limit;
5 end

```

Listing 15: Checks if a single line's voltage drop complies with REBT limits.

D.16 F4b_checkInstallationCompliance.m

```

1 function F4b_checkInstallationCompliance(results, U_source)
2 fprintf('\n--- REBT COMPLIANCE CHECK ---\n');
3 total_drop = U_source - results{end}.voltage_out;
4 fprintf('Total Voltage Drop: %.2f V (%.2f%%)\n', total_drop, (total_drop
   /U_source)*100);
5 end

```

Listing 16: Checks a multi-segment installation against tiered REBT limits.

D.17 F5_plotVoltageProfile.m

```

1 function [] = F5_plotVoltageProfile(U_source, deltaU_total_ac,
   total_length, circuitType)
2 length_vector = linspace(0, total_length, 200);
3 voltage_profile_ac = U_source - (deltaU_total_ac / total_length) *
   length_vector;
4 if strcmp(circuitType, 'lighting'), limit_percent = 4.5/100; else,
   limit_percent = 6.5/100; end
5 min_voltage_limit = U_source * (1 - limit_percent);
6 figure; hold on;
7 plot(length_vector, voltage_profile_ac, 'r-', 'LineWidth', 2, '
   DisplayName', 'Voltage Profile (AC Res.)');
8 line([0, total_length], [min_voltage_limit, min_voltage_limit], 'Color',
   'g', 'LineStyle', '--', 'DisplayName', sprintf('REBT Limit (%.2f V)',
   min_voltage_limit));
9 title('Voltage Profile Along the Line'); xlabel('Distance from Source (m
   )'); ylabel('Voltage (V)');
10 grid on; box on; legend('Location', 'southwest');
11 ylim([min_voltage_limit - (0.02*U_source), U_source * 1.02]);
12 hold off;
13 end

```

Listing 17: Plots the voltage profile for a single line.

D.18 F6_plotInstallationProfile.m

```

1 function F6_plotInstallationProfile(results, U_source)
2 numSegments = length(results);
3 node_voltages = [U_source, cellfun(@(x) x.voltage_out, results)'];
4 node_lengths = [0, cumsum(cellfun(@(x) x.length, results))'];
5 segment_names = cellfun(@(x) x.name, results, 'UniformOutput', false);
6 figure; hold on;
7 stairs(node_lengths, node_voltages, 'b-', 'LineWidth', 2, 'DisplayName',
   'Voltage Profile');
8 plot(node_lengths, node_voltages, 'ro', 'MarkerFaceColor','r', '
   HandleVisibility','off');
9 limit_total_percent = 5.0 / 100; % Simplified for example
10 min_voltage_limit = U_source * (1 - limit_total_percent);
11 line([0, node_lengths(end)], [min_voltage_limit, min_voltage_limit], '
   Color','k', 'LineStyle','--', 'DisplayName', sprintf('Overall REBT
   Limit (%.2f V)', min_voltage_limit));
12 title('Voltage Profile Across Complete Installation');
13 xlabel('Cumulative Line Length (m)'); ylabel('Voltage (V)');
14 grid on; box on; legend('show', 'Location', 'southwest');
15 for i = 1:numSegments
16     x_pos = node_lengths(i) + (node_lengths(i+1) - node_lengths(i))/2;

```

```

17     y_pos = node_voltages(i+1) + 5;
18     text(x_pos, y_pos, strrep(segment_names{i}, '_', ' '), '
    HorizontalAlignment', 'center', 'BackgroundColor', 'w');
19 end
20 hold off;
21 end

```

Listing 18: Plots the voltage profile across a multi-segment installation.

D.19 G1_ConductorSizingAnalysis.m

```

1 function G1_ConductorSizingAnalysis()
2 sigma_aluminum = 36; LCC_years = 20; hours_per_year = 4000; cost_per_kWh
    = 0.15;
3 U_source = 400; lineLength = 200; loadCurrent = 150; cos_phi = 0.85;
4 material = 'Aluminum';
5 [~, ~, sections_data] = G3_selectStandardSection(0, material);
6 s_required_vd = G2_calculateRequiredSection('three-phase', lineLength,
    loadCurrent, cos_phi, sigma_aluminum, U_source * 0.05);
7 s_technical_data = G3_selectStandardSection(s_required_vd, material);
8 s_technical = s_technical_data.section;
9 sections_to_analyze = sections_data(arrayfun(@(x) x.section,
    sections_data) >= s_technical);
10 num_sections = numel(sections_to_analyze);
11 total_costs=zeros(1,num_sections); cable_costs=zeros(1,num_sections);
    loss_costs=zeros(1,num_sections);
12 for i = 1:num_sections
13     [total_costs(i), cable_costs(i), loss_costs(i)] =
        G4_calculateLifecycleCost(sections_to_analyze(i), 'three-phase',
        lineLength, loadCurrent, sigma_aluminum, LCC_years, hours_per_year,
        cost_per_kWh);
14 end
15 [~, optimal_idx] = min(total_costs);
16 s_optimal = sections_to_analyze(optimal_idx).section;
17 G5_plotEconomicAnalysis(sections_to_analyze, cable_costs, loss_costs,
    total_costs, s_technical, s_optimal);
18 fprintf('--- SIZING RESULTS ---\n');
19 fprintf('Technically Compliant Section: %d mm^2\n', s_technical);
20 fprintf('Economically Optimal Section: %d mm^2\n', s_optimal);
21 end

```

Listing 19: Main script for conductor sizing and economic analysis.

D.20 G2_calculateRequiredSection.m

```

1 function s = G2_calculateRequiredSection(lineType, d, I, cos_phi, sigma,
    deltaU_max)
2 if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else,
    phase_factor = 2; end
3 s = (phase_factor * d * I * cos_phi) / (sigma * deltaU_max);
4 end

```

Listing 20: Calculates the minimum required cross-section based on voltage drop.

D.21 G3_selectStandardSection.m

```
1 function [selected_section, ~, standard_sections_data] =  
    G3_selectStandardSection(s_required, material)  
2 if strcmpi(material, 'Copper'), data = [1.5,24,0.5; 2.5,32,0.8;  
    4,42,1.2; 6,54,1.7; 10,73,2.8; 16,98,4.5; 25,129,7.0; 35,158,9.5;  
    50,188,13.0; 70,232,18.0; 95,275,24.0; 120,315,30.0];  
3 else, data = [16,76,3.0; 25,100,4.5; 35,123,6.0; 50,146,8.0;  
    70,180,11.5; 95,215,15.0; 120,245,19.0; 150,280,23.0]; end  
4 standard_sections_data = struct('section', num2cell(data(:,1)), '  
    ampacity', num2cell(data(:,2)), 'cost_per_meter', num2cell(data(:,3))  
    );  
5 idx = find(data(:,1) >= s_required, 1, 'first');  
6 if isempty(idx), selected_section = standard_sections_data(end);  
7 else, selected_section = standard_sections_data(idx); end  
8 end
```

Listing 21: Selects the next available standard conductor section from a table.

D.22 G4_calculateLifecycleCost.m

```
1 function [total_cost, cable_cost, loss_cost] = G4_calculateLifecycleCost  
    (section_data, lineType, d, I, sigma, years, hours_per_year,  
    cost_per_kWh)  
2 s = section_data.section; cost_per_meter = section_data.cost_per_meter;  
3 if strcmp(lineType, 'three-phase'), num_conductors = 3; else,  
    num_conductors = 2; end  
4 cable_cost = d * cost_per_meter * num_conductors;  
5 total_resistance = (1/sigma) * d / s;  
6 power_loss_watts = num_conductors * (I^2) * total_resistance;  
7 total_kWh_lost = (power_loss_watts / 1000) * hours_per_year * years;  
8 loss_cost = total_kWh_lost * cost_per_kWh;  
9 total_cost = cable_cost + loss_cost;  
10 end
```

Listing 22: Calculates the lifecycle cost (initial + losses) for a given section.

D.23 G5_plotEconomicAnalysis.m

```
1 function [] = G5_plotEconomicAnalysis(sections, cable_costs, loss_costs,  
    total_costs, s_technical, s_optimal)  
2 figure;  
3 section_labels = arrayfun(@(x) num2str(x.section), sections, '  
    UniformOutput', false);  
4 x_categories = categorical(section_labels);  
5 x_categories = reordercats(x_categories, section_labels);  
6 bar_data = [cable_costs', loss_costs'];  
7 bar(x_categories, bar_data, 'stacked');  
8 hold on;  
9 plot(x_categories, total_costs, 'd-r', 'LineWidth', 2, 'MarkerFaceColor'  
    , 'r', 'MarkerSize', 8, 'DisplayName', 'Total Lifecycle Cost');  
10 optimal_idx = find([sections.section] == s_optimal);  
11 if ~isempty(optimal_idx)  
12     text(x_categories(optimal_idx), total_costs(optimal_idx), ' Optimal'  
    , 'Color', 'red', 'FontWeight', 'bold');
```

```

13 end
14 technical_idx = find([sections.section] == s_technical);
15 if ~isempty(technical_idx)
16     text(x_categories(technical_idx), total_costs(technical_idx), '
        Technical Min.', 'Color', 'black', 'VerticalAlignment', 'top');
17 end
18 title('Economic Analysis of Conductor Sizing');
19 xlabel('Standard Conductor Section (mm^2)');
20 ylabel('Total Lifecycle Cost (EUR)');
21 legend({'Initial Cable Cost', 'Cost of Energy Losses', 'Total Lifecycle
        Cost'}, 'Location', 'northoutside', 'Orientation', 'horizontal');
22 grid on; hold off;
23 end

```

Listing 23: Plots the economic analysis results to find the optimal section.

D.24 G6_verifyFinalDesign.m

```

1 function final_vd_percent = G6_verifyFinalDesign(lineType, d, I, cos_phi
    , sigma, s_final, U_source)
2 if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else,
    phase_factor = 2; end
3 deltaU_volts = (phase_factor * d * I * cos_phi) / (sigma * s_final);
4 final_vd_percent = (deltaU_volts / U_source) * 100;
5 end

```

Listing 24: Performs a final verification of the voltage drop for the chosen design.